

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ» (ФГБОУ ВО
«ВВГУ»)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ КАФЕДРА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
по дисциплине
«Информатика и основы программирования»

Студент
гр. БИН-25-3

С.Д. Мурадуллаева

Ассистент
преподавателя

М.В. Водяницкий

Владивосток 2025

Задание

Выполнить задания на Python и оформить отчет по стандартам ВВГУ.

Задание 1. Дан список из 10 различных целых чисел. Необходимо найти в нем число 3 и заменить на 30.

Задание 2. Дан список из 5 целых чисел. Необходимо превратить его в список квадратов этих чисел.

Задание 3. Имеется список различных целых чисел. Программа должна найти наибольшее из чисел списка и разделить его на длину списка.

Задание 4. Имеется кортеж из нескольких произвольных элементов. Необходимо этот кортеж отсортировать. Если хотя бы один элемент не является числом, то кортеж остается неизменным.

Задание 5. Имеется словарь товаров в магазине. Необходимо найти товар с минимальной и максимальной ценой.

Задание 6. Имеется список произвольных элементов. Необходимо на основе этого списка создать словарь, где каждый элемент списка будет и ключом, и значением.

Задание 7. Имеется словарь перевода английских слов на русский, где ключ английского слово, значение - русского. Необходимо реализовать программу которая получает на ввод русское слово и результатом выдает перевод на английский.

Задание 8. Реализовать игру Камень-Ножницы-Бумага-Ящерица-Спок. Программа должна запрашивать у пользователя ввод одного из вариантов. Второй вариант случайно генерирует сама программа и возвращает победителя.

Правила игры следующие:

- Ножницы режут бумагу
- Бумага покрывает камень
- Камень давит ящерицу
- Ящерица отравляет Спока
- Спок ломает ножницы
- Ножницы обезглавливают ящерицу
- Ящерица съедает бумагу
- Бумага подставляет Спока
- Спок испаряет камень
- Камень разбивает ножницы

Задание 9. Дан список слов - например:

["яблоко", "груша", "банан", "киви", "апельсин", "ананас"]

Необходимо создать новый словарь, где:

- Ключом будет первая буква слова

- Значением - список всех слов, начинающихся с этой буквы

Пример результата:

```
python
{'я': ['яблоко'], 'г': ['груша'], 'б': ['банан'], 'к': ['киви'], 'а': ['апельсин', 'ананас']}
```

Задание 10. Дан список кортежей, где каждый кортеж содержит имя студента и его оценки, например:

```
python
[("Анна", [5, 4, 5]), ("Иван", [3, 4, 4]), ("Мария", [5, 5, 5])]
```

Необходимо:

1. Создать словарь, где ключ - имя студента, значение - его средняя оценка
2. Найти студента с наибольшей средней оценкой и вывести его имя и средний балл

Пример результата:

```
txt
Мария имеет наивысший средний балл: 5.0
```

Содержание

1	Выполнение работы	5
1.1	Задание 1	5
1.2	Задание 2	5
1.3	Задание 3	5
1.4	Задание 4	6
1.5	Задание 5	6
1.6	Задание 6	7
1.7	Задание 7	7
1.8	Задание 8	8
1.9	Задание 9	9
1.10	Задание 10	9

1 Выполнение работы

1.1 Задание 1

Цель: Научиться находить и заменять элементы в списке.

```
numbers = [5, 12, 3, 8, 15, 7, 3, 20, 1, 9]

for i in range(len(numbers)):
    if numbers[i] == 3:
        numbers[i] = 30

print("Результат:", numbers)
```

Рисунок 1 – Листинг программы для задания 1

Описание решения:

Создается список из 10 целых чисел. С помощью цикла for перебираются все элементы списка. Если встречается число 3, оно заменяется на 30. Используется функция range(len()) для получения индексов элементов.

1.2 Задание 2

Цель: Освоить преобразование списков с использованием list comprehension.

```
numbers = [2, 5, 8, 3, 7]
squares = [x**2 for x in numbers]

print("Исходный список:", numbers)
print("Квадраты чисел:", squares)
```

Рисунок 2 – Листинг программы для задания 2

Описание решения:

Используется синтаксис list comprehension [x**2 for x in numbers], который создает новый список, содержащий квадраты каждого элемента исходного списка.

1.3 Задание 3

Цель: Научиться находить максимальный элемент и работать с длиной списка.

```
numbers = [12, 45, 23, 67, 34, 89, 56]
max_number = max(numbers)
result = max_number / len(numbers)

print(f"Список: {numbers}")
print(f"Максимальное число: {max_number}")
print(f"Длина списка: {len(numbers)}")
```

```
print(f"Результат деления: {result:.4f}")
```

Рисунок 3 – Листинг программы для задания 3

Описание решения:

Находим максимальный элемент с помощью `max()`, длину списка с помощью `len()`, затем выполняем деление и форматируем вывод до четырех знаков после запятой.

1.4 Задание 4

Цель: Изучить обработку исключений при работе с кортежами.

```
def sort_tuple(t):
    try:
        return tuple(sorted(t))
    except TypeError:
        return t

# Тестирование
tuple1 = (5, 2, 8, 1, 9)
tuple2 = (3, "hello", 7, 4)

print("Кортеж 1 до сортировки:", tuple1)
print("Кортеж 1 после сортировки:", sort_tuple(tuple1))
print("Кортеж 2 до сортировки:", tuple2)
print("Кортеж 2 после сортировки:", sort_tuple(tuple2))
```

Рисунок 4 – Листинг программы для задания 4

Описание решения:

Используется блок `try-except` для попытки сортировки кортежа. Если возникает `TypeError` (при наличии нечисловых элементов), возвращается исходный кортеж.

1.5 Задание 5

Цель: Научиться анализировать данные в словарях.

```
products = {
    "хлеб": 50,
    "молоко": 80,
    "сыр": 300,
    "яблоки": 120,
    "кофе": 450,
    "чай": 200
}

min_price = min(products.values())
max_price = max(products.values())
```

```

min_products = [name for name, price in products.items() if price == min_price]
max_products = [name for name, price in products.items() if price == max_price]

print("Товары с минимальной ценой:", min_products, f"({min_price} руб.)")
print("Товары с максимальной ценой:", max_products, f"({max_price} руб.)")

```

Рисунок 5 – Листинг программы для задания 5

Описание решения:

Используем min() и max() для нахождения минимальной и максимальной цены. Затем с помощью list comprehension находим все товары с соответствующей ценой.

1.6 Задание 6

Цель: Освоить создание словарей с использованием dictionary comprehension.

```

elements = ["apple", "banana", "cherry", "date", "labubu"]
result_dict = {element: element for element in elements}

print("Исходный список:", elements)
print("Результирующий словарь:", result_dict)

```

Рисунок 6 – Листинг программы для задания 6

Описание решения:

Каждый элемент списка становится одновременно ключом и значением в новом словаре. Dictionary comprehension аналогично list comprehension позволяет компактно создавать словари.

1.7 Задание 7

Цель: Реализовать обратный поиск в словаре.

```

eng_rus_dict = {
    "apple": "яблоко",
    "banana": "банан",
    "cat": "кот",
    "dog": "собака",
    "house": "дом"
}

rus_word = input("Введите русское слово: ")

rus_eng_dict = {russian: english for english, russian in eng_rus_dict.items()}

if rus_word in rus_eng_dict:
    print(f"Перевод на английский: {rus_eng_dict[rus_word]}")
else:

```

```
print("Слово не найдено в словаре")
```

Рисунок 7 – Листинг программы для задания 7

Описание решения:

Создаем англо-русский словарь. Создаем обратный словарь с помощью dictionary comprehension, где русские слова становятся ключами, а английские - значениями. Ищем введенное слово в обратном словаре. Выводим результат или сообщение об ошибке.

1.8 Задание 8

Цель: Реализовать логическую игру с использованием словарей.

```
import random

choices = ["камень", "ножницы", "бумага", "ящерица", "спок"]

rules = {
    "камень": ["ножницы", "ящерица"],
    "ножницы": ["бумага", "ящерица"],
    "бумага": ["камень", "спок"],
    "ящерица": ["бумага", "спок"],
    "спок": ["камень", "ножницы"]
}

print("Доступные варианты:", ", ".join(choices))
user_choice = input("Ваш выбор: ").lower()

if user_choice not in choices:
    print("Неверный выбор!")
else:
    computer_choice = random.choice(choices)
    print(f"Компьютер выбрал: {computer_choice}")

    if user_choice == computer_choice:
        print("Ничья!")
    elif computer_choice in rules[user_choice]:
        print("Вы победили!")
    else:
        print("Компьютер победил!")
```

Рисунок 8 – Листинг программы для задания 8

Описание решения:

Правила игры хранятся в словаре, где для каждого варианта указано, какие другие варианты он побеждает. Пользователь делает выбор. Компьютер делает случайный выбор с помощью

random.choice(). Проверяем условия победы согласно правилам. Выводим соответствующий результат.

1.9 Задание 9

Цель: Научиться группировать данные по определенному признаку.

```
words = ["яблоко", "груша", "банан", "киви", "апельсин", "ананас", "арбуз"]

result = {}
for word in words:
    first_letter = word[0]
    if first_letter not in result:
        result[first_letter] = []
    result[first_letter].append(word)

print("Исходный список слов:", words)
print("Сгруппированный словарь:", result)
```

Рисунок 9 – Листинг программы для задания 9

Описание решения:

Для каждого слова извлекается первая буква. Если этой буквы еще нет в словаре-результате, создается новая запись с пустым списком. Слово добавляется в соответствующий список. Выводим исходные данные и результат.

1.10 Задание 10

Цель: Освоить анализ данных с вычислением статистических показателей.

```
students = [
    ("Анна", [5, 4, 5, 4]),
    ("Иван", [3, 4, 4, 3]),
    ("Мария", [5, 5, 5, 5]),
    ("Петр", [4, 3, 5, 4])
]

# словарь средних оценок
average_grades = {}
for name, grades in students:
    average = sum(grades) / len(grades)
    average_grades[name] = average

# студент с макс ср баллом
best_student = max(average_grades, key=average_grades.get)
best_grade = average_grades[best_student]

print("Средние оценки студентов:")
for student, avg in average_grades.items():
    print(f"{student}: {avg:.2f}")
```

```
print(f"\n{best_student} имеет наивысший средний балл: {best_grade:.2f}")
```

Рисунок 10 – Листинг программы для задания 10

Описание решения:

Для каждого студента вычисляется средняя оценка. Результаты сохраняются в словарь. Затем с помощью функции `max()` с `key`-параметром находится студент с наибольшим средним баллом. Выводим все средние оценки с точностью до 2-х знаков после запятой и выделяем лучшего студента.